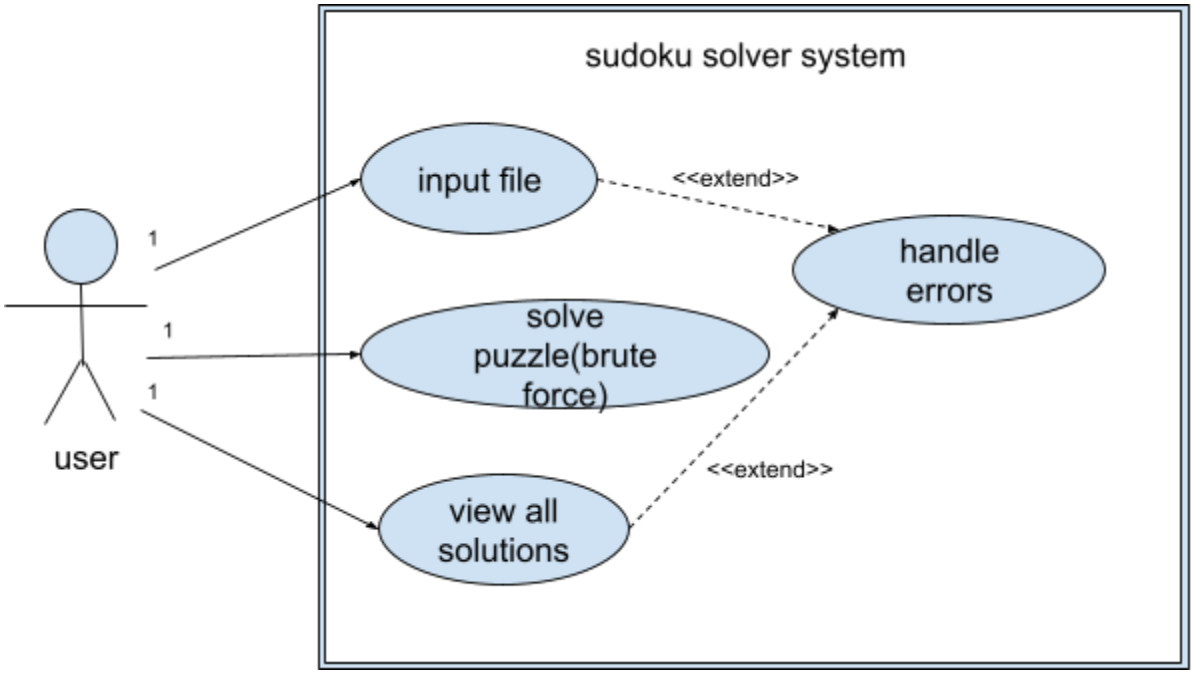


USE CASES:

1. List of use cases in system:
    a. input file
    b. solve puzzle
    c. view all solutions
    d. (optional use case) handle errors
2. brief description:

    Input file

    System is to prompt the user for a .txt file. this file should contain an unfinished 9x9 sudoku prompt. The system can then create a 9x9 matrix and parse the text file and input each character into the matrix.

    Solve Puzzle (all combinations)

    The System takes the 9x9 matrix of the unfinished sudoku board and goes over every empty square, checking all the values that validate the rules (no duplicates in same row, column, or 3x3 grid) then backtracks to check every combination, even if a solution was found

    View all solutions

    When the puzzle is fully solved, the system prints out the completed matrix in the same format as the inputted file, with the solved attempt number at the top. after the recursive function completes, the total amount of solutions is printed, or error messages will be printed if there are no possible solutions.

    (optional) handle errors

    the system will not run recursive functions if the board is inherently unsolveable or already completed, if the text file is unopenable, or not in a 9x9 format. the system will give proper error messages and terminate.
3. outline:

Input File:

- user starts executable code
- system prepares a variable to store a file
- system asks user for input of a file located in the same folder
- user inputs .txt file (for example in our case puzzle1.txt) and hits enter
- the system establishes a read file stream for the text file
- the system puts aside 81 bits of memory for the board and maps them into the matrix
- the system adds in every value from the text file including the underscores.
- the system ends the read file stream
- the system checks the file for validity
- the system finds the first empty square in the matrix (read from left to right, top row to bottom row)
- the system pushes the matrix to the recursive solving program with the first empty square marked

  EXTENSIONS:

  case a. file is unreadable
    - the system cannot find or read the inputted file.
    - the system informs the user of this
    - the system terminates immediately after

  case b. the .txt file contains an irregular matrix size
    - the system's read file stream ends too early or contains unexpected items
    - the system informs the user and explains it expects a 9x9 grid
    - the system terminates immediately after

Solve Puzzle

- starting on the first empty item in the matrix, the system uses a loop to try every valid value 1-9.
- the system checks if a value is found in the same row, column, or 3x3 grid. if not, it is a valid integer.
- the system commits the valid integer to the board.
- the system finds the next empty tile using a while loop.
- the system calls its own recursive call at the next empty box.
- the system reaches the last cell to fill
- the system sends the matrix to the printing function to print it out
- the system removes the digit in the cell its on to try other values.
- when all values have been tried, change the digit back into a blank '_' and backtrack to try values at previous squares.

View All Solutions

- the (printboard) system recieves a fully complete board.
- the system increases the glocal solution counter by one.
- the system prints a header for the matrix and its given iteration number
- the system prints the matrix the same way it was received: one space apart, no trailing spaces
- the system prints an extra whitespace line to separate board solutions

EXTENSIONS:

- the system completes with the global solution counter at 0.
- the system prints that no solutions have been found and terminates.

4. Detailed descriptions

Input File ● The system is started and asks the user for an input file. ● when received, it attempts to read the file and binds to an std::ifstream buffer. ● the system then assigns the expected 81 values (1-9 or _) to a 9x9 matrix structure, ignoring spaces and newlines. ● the system then closes the stream and the matrix is available for use. ● in the case where the matrix is unreadable or there are an unexpected amount of characters, the system informs the user and terminates. ● the system additionally finds the first empty in the matrix for the solve puzzle use case to use.
Solve Puzzle ● The solve puzzle use case is the part that brute-forces every possible combination in the sudoku board. ● starting at the first empty (or underscore _) in the matrix, the system tests values 1-9. ● if the value is valid in that box (there are no duplicates in the same row, column, or respective 3x3 grid), the puzzle inputs it. ● the system checks for the next empty square and calls the recursive function there every time a valid integer is inserted. ● when all valid integers have been tried in that box, returns the box to _ and backtracks to the previous function to try more values in previous boxes. ● the system ends when the first called function fully completes (which means all recursive functions have to complete).
View All Solutions ● This use case formats and displays all found solutions to the matrix. ● if at any point the board is full, meaning the system found a solution,the system increases our solution counter and the matrix is given to a function that properly loops though the entire matrix and prints each item exactly one space apart with no leading spaces. ● the system also separates them with a newline by each row. ● in the case that no solution was found, or the solution counter is zero, the system prints that there were no solutions and does not attempt to print the matrix.